

M06 / LEARNER BOOK

Linux, Git and reproducible projects

Standalone Zero-to-Job-Ready Data Engineer 2026

13 controlled lessons

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

START HERE How to use this module	3
DE06-01 Verify WSL2 and choose a safe Linux workspace	4
DE06-02 Linux navigation and safe file operations	7
DE06-03 Pipes, redirection, command history and inspection	10
DE06-04 Permissions, packages, sudo and command safety	12
DE06-05 Git state: init, status, add, commit, diff and log	14
DE06-06 Branches, merges and real conflict resolution	16
DE06-07 GitHub remotes: push, fetch, pull and tracking	18
DE06-08 Pull requests and review workflow	20
DE06-09 Python virtual environments inside a repository	22
DE06-10 .gitignore, secrets and repository hygiene	24
DE06-11 restore, revert, reflog and safe history repair	26
DE06-12 Shell scripts, exit codes and environment-driven commands	29
DE06-13 Repository handoff and clean-clone reproducibility	31

START HERE

How to use this module

Open the matching practice folder from the course root: 04_M06/practice. Work on a copy. Commands printed in a lesson use that module as the working folder unless the lesson names a different location.

Study in lesson order. After the last lesson, use Quick Revision, take the Module Revision Test, then open its separate Protected Answers. Gates are additional assessments at the boundaries shown in the course guide.

This book is written for a beginner. Work through the lessons in order. Read the new words, watch the assigned real instructor/source range when useful, perform the guided example on your laptop, then save an independent attempt before opening the explained review. Ready-made revision is included; you do not need to rewrite the book as notes.

Study directly from the learner book and practice files. Keep your own attempts, observed outputs and result evidence. Review material is for diagnosis after a genuine attempt, not for copying. Changed retries are separate evidence. There is no fixed time-to-job promise.

M06 is where your work stops being a collection of files on one Windows laptop and starts behaving like engineering work that another person can inspect, reproduce and review. You will use Linux through WSL, but this is not a Linux-administrator course. You will use Git and GitHub, but this is not a collection of commands to memorize. The target is a working mental model: where you are, what state your files are in, what Git has recorded, what exists only on your machine, what exists on a remote, and how a second person can reproduce the project without hidden knowledge.

The module deliberately keeps Docker, Airflow and cloud out. Those come later. Here you learn the operating-system and version-control habits that make those later tools much less confusing. Every destructive or history-changing command is taught with a safety boundary. Every collaboration concept is tied to observable state rather than screenshots alone.

DE06-01

Verify WSL2 and choose a safe Linux workspace

Watch first

[1. MIT Missing Semester - Course overview and the shell \(embedded lecture\)](#)

Optional visual - no verified bounded range. Use the shell navigation and permissions demonstrations. WSL installation itself follows the Windows setup instructions in this lesson.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Source: Microsoft Learn - Install WSL URL: <https://learn.microsoft.com/en-us/windows/wsl/install>
Use: [Current install/verify guidance](#); prerequisites, `wsl --install`, `wsl --list --verbose`, WSL 2. Use only for setup truth; the course owns workspace practice.

Why this matters

A beginner can have Python installed and still be blocked by a sentence such as "run this in Linux," "use Bash," or "clone it inside WSL." Windows and Linux are not two mysterious competing computers. WSL gives you a Linux environment on your Windows machine. For this course, the important skill is knowing which environment and filesystem you are using before you install packages, create a repository or run a command.

Picture it in your head

Think of your laptop as one house with two workrooms. Windows has paths such as `C:\Users\...`. Your Linux distribution has a Linux home such as `/home/ashish`. WSL can see Windows files through mounts such as `/mnt/c/...`, but that does not mean every project should live there. For Linux-heavy developer tools, keeping the working repository inside the Linux filesystem (`~/de-study/...`) usually gives a simpler Unix permission/path model. Files you merely want to exchange with Windows can still be reached through `/mnt/c`.

WSL is the Windows feature. WSL 2 is its current virtualized architecture. Ubuntu is a Linux distribution you may run inside it. Bash is a shell that accepts commands inside Linux. These are different layers; saying "I opened WSL" is often shorthand for "I opened my Ubuntu shell under WSL."

Follow with me once

First, in Windows PowerShell, inspect rather than reinstall blindly:

```
wsl --status
```

```
wsl --list --verbose
```

If WSL is not installed, Microsoft's current route is an Administrator PowerShell with `wsl --install`, followed by a restart. Do not keep rerunning install commands if WSL is already present. The exact current Microsoft page is linked above because setup behavior can change.

Then open your Linux distribution and run:

```
whoami
uname -a
pwd
printf 'HOME=%s\n' "$HOME"
ls -la
```

pwd answers "where am I?" It does not mean "show all files." \$HOME is your Linux user's home directory. Create the course workspace only after you have proved where you are:

```
mkdir -p ~/de-study/M06
cd ~/de-study/M06
pwd
```

Expected shape: /home/<your-linux-user>/de-study/M06, not a random Windows download path and not /root.

Now deliberately look at the Windows mount without moving your project there:

```
ls /mnt/c/Users
```

You have just proved the boundary instead of guessing it.

A failure worth understanding

If wsl --list --verbose shows version 1 for the distribution, that is not the same as "WSL is broken." Record the actual state and use Microsoft's current upgrade guidance. If the Linux prompt says you are root, stop and understand why before building a learner workspace there; ordinary study should run as your normal Linux user.

If a command works in PowerShell but not Bash, or vice versa, ask first: which shell am I in? Do not solve that by randomly reinstalling Python/Git in both places.

Try it yourself

Create ~/de-study/M06/DE06-01-first-attempt/. Save a short environment.txt containing

the observed Linux username, pwd, kernel/WSL information, and a one-sentence statement of where your future M06 repositories will live. Also write one Windows path and its WSL /mnt/c/... equivalent.

Check your work

You can explain WSL vs WSL2 vs Ubuntu vs Bash; prove whether your distro is WSL2; identify your Linux home; and distinguish a Linux path from the mounted Windows path. You have not claimed that installation succeeded unless you actually observed it on your PC.

Changed retry

Without rereading your first commands, open a fresh Linux shell, navigate from \$HOME to a new ~/de-study/M06-retry folder, create it if missing, and state whether C:\Users\YourName\Desktop is a valid Bash path. Then show the corresponding /mnt/c/... form.

Evidence to keep

Prove WSL/distro/workspace state on your own PC; save observed commands and choose a Linux-home M06 workspace. Preserve the first attempt, observed output/error, one independent validation, and your own-words teach-back. If review is used, keep the changed retry separately.

DE06-02

Linux navigation and safe file operations

Watch first

[1. freeCodeCamp - Command Line Basics for Beginners \(2026\)](#)

02:56-30:18: command line mental model, ls, navigation, touch/rm, mkdir/rmdir.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Source: freeCodeCamp - Command Line Basics for Beginners (2026) URL:

<https://www.youtube.com/watch?v=mABpAl-pCw0> Use: 02:56-30:18: command line mental model, ls, navigation, touch/rm, mkdir/rmdir.

Why this matters

Data engineering scripts constantly create folders, inspect files, move outputs and clean temporary work. The danger is not that `cd` is difficult. The danger is acting in the wrong directory or using a destructive command before you have inspected what it will affect.

Picture it in your head

Your terminal always has a current working directory. Relative paths start from that point; absolute paths start from `/`. If you say `data/orders.csv`, Linux resolves it relative to where you currently are. If you say `/home/ashish/de-study/M06/data/orders.csv`, the starting point is explicit.

Imagine a warehouse worker with a trolley. `pwd` tells the worker which aisle they stand in. `ls` inspects the shelf. `cd` moves to another aisle. `cp` makes another item. `mv` changes its location or name. `rm` destroys the named item; there is no ordinary recycle-bin promise in a shell workflow.

Follow with me once

Copy the supplied data folder into your learner-owned sandbox, then work only there. Use a fresh directory such as:

```
mkdir -p ~/de-study/M06/linux-sandbox
cd ~/de-study/M06/linux-sandbox
pwd
```

Create a small tree:

```
mkdir -p raw processed evidence
printf 'batch_id=B001\n' > raw/batch.txt
touch evidence/notes.txt
find . -maxdepth 2 -type f -print
```

Before every move, predict the resulting path:

```
cp raw/batch.txt processed/batch_copy.txt
mv processed/batch_copy.txt processed/B001.txt
ls -l raw processed
```

The original raw/batch.txt still exists because cp copied. mv renamed the copy; it did not create a second one.

Try paths deliberately:

```
cd raw
cat batch.txt
cat ../processed/B001.txt
cd ..
```

.. means the parent directory. . means the current directory. ~ expands to your Linux home.

Now inspect before deleting:

```
ls -l processed/B001.txt
rm processed/B001.txt
ls -l processed
```

This is safe because you created the sandbox and inspected the exact file first. Do not practice rm -rf on your home directory, a course master folder or a path assembled from an unverified variable.

The space-in-a-name trap

This command has two arguments, not one filename:

```
cat monthly report.txt
```

If the file is really named monthly report.txt, quote it:

```
cat 'monthly report.txt'
```

Better engineering habit: prefer simple predictable names in project automation so quoting bugs are less likely.

Try it yourself

Inside a new DE06-02-first-attempt folder, create incoming, archive, and evidence. Create two tiny files, copy one to archive, rename the other, then move it. Save a terminal transcript or typed record that shows pwd before and after. Explain which commands changed content, which changed location/name, and which merely inspected state.

Check your work

You can navigate without repeatedly opening File Explorer, predict relative paths, distinguish copy from move, and show exactly what an rm command will target before you run it.

Changed retry

Start from \$HOME, create a different tree M06-retry/source and M06-retry/ready, then copy notes.txt using an absolute source path and relative destination path. Prove both files exist afterward. Delete only the retry copy after inspecting it.

Evidence to keep

Build a safe folder tree, copy/move/rename files, and explain relative vs absolute paths before deleting a disposable file. Preserve the first attempt, observed output/error, one independent validation, and your own-words teach-back. If review is used, keep the changed retry separately.

DE06-03

Pipes, redirection, command history and inspection

Watch first

[1. freeCodeCamp - Command Line Basics for Beginners \(2026\)](#)

30:18-43:32: echo/cat and file inspection. Course adds pipes, stderr, grep, wc, head/tail and validation.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Source: freeCodeCamp - Command Line Basics for Beginners (2026) URL:

<https://www.youtube.com/watch?v=mABpAl-pCw0> Use: 30:18-43:32: echo/cat and file inspection. Course adds pipes, stderr, grep, wc, head/tail and validation.

Why this matters

A terminal becomes powerful when commands can be connected. Instead of manually opening a 50,000-line log, you can read a stream, filter it, count it and send a result to another file. The same idea later appears in data pipelines: one stage emits output that another stage consumes.

Picture it in your head

A normal command has three standard channels: stdin (input), stdout (normal output), and stderr (error output). A pipe | sends stdout from the command on the left into stdin of the command on the right. > writes stdout to a file and overwrites that file. >> appends. 2> redirects stderr.

Do not memorize symbols as punctuation. Ask: "Which stream is moving where?"

Follow with me once

Work on a copy of data/pipeline.log and data/linux_orders.csv:

```
cat pipeline.log
head -n 2 pipeline.log
tail -n 2 pipeline.log
wc -l pipeline.log
grep 'WARN' pipeline.log
```

Now connect stages:

```
grep 'INFO' pipeline.log | wc -l
```

The first command emits matching INFO lines. The pipe feeds those lines to wc -l, which counts them. If you later change the file, the answer changes because the command is operating on current data.

Create evidence safely:

```
grep 'WARN' pipeline.log > warnings.txt
cat warnings.txt
grep 'INFO' pipeline.log >> warnings.txt
```

After `>`, `warnings.txt` contains the new output instead of its previous contents. After `>>`, new output is added to the end. This distinction matters when a pipeline output must be deterministic: accidental append can duplicate prior-run output.

Now separate errors:

```
cat file_that_does_not_exist.txt 2> error.txt
cat error.txt
```

The command failed, but the error was captured from `stderr`. In real automation you should also inspect the command's exit code, which comes in DE06-12.

For CSV inspection, do not pretend `grep` is a CSV parser. It is useful for quick textual inspection, not for safely parsing quoted commas. You already learned proper CSV parsing in M05. Use terminal tools for questions suited to text streams, and Python for structured parsing.

History as evidence, not magic

Use `history` | `tail` to inspect recent commands on shells that support it. Shell history can help reproduce what you typed, but it is not a reliable project README: histories can be cleared, contain secrets, and differ across machines. Promote important commands into documented scripts or README instructions.

Try it yourself

Using your own copy of `pipeline.log`, produce three files: `warn_only.txt`, `info_count.txt`, and `missing_file_error.txt`. Before each redirection, state whether the operation should overwrite or append. Then answer: why would `>>` be dangerous in a daily report that is supposed to be regenerated from scratch?

Check your work

You can trace a pipeline left-to-right, distinguish `stdout` from `stderr`, show the difference between `>` and `>>`, and refuse to use `grep` as a substitute for structured CSV parsing.

Changed retry

On a new small text file, count only lines containing `Completed`, save the count to a fresh output file, deliberately run one failing `cat`, redirect only its `stderr`, and prove the normal count file was not polluted by the error message.

Evidence to keep

Use pipes/redirection to create filtered/count/error evidence while distinguishing `stdout`, `stderr`, overwrite and append. Preserve the first attempt, observed output/error, one independent validation, and your own-words teach-back. If review is used, keep the changed retry separately.

DE06-04

Permissions, packages, sudo and command safety

Watch first

[1. MIT Missing Semester - Course overview and the shell \(embedded lecture\)](#)

Optional visual - no verified bounded range. Use the shell navigation and permissions demonstrations. WSL installation itself follows the Windows setup instructions in this lesson.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Source: DigitalOcean - Essential Linux Commands (current reference) URL:

<https://www.digitalocean.com/community/tutorials/linux-commands> Use: Use only the

sudo/chmod/chown/permission sections as current command reference. The learner book remains authoritative for least-privilege reasoning and the safe practice task.

Why this matters

Linux permissions are one of the first places where beginners solve a small problem with a dangerously large hammer. "Permission denied" does not automatically mean "use sudo," and chmod 777 is not a professional universal fix. You need enough permission reasoning to work safely without becoming a Linux administrator.

Picture it in your head

A typical ls -l line begins with permission bits for owner, group, and others. The letters are r read, w write, and x execute. A regular shell script can contain perfect text and still refuse ./script.sh if the execute bit is not set.

sudo means "run this command with elevated privileges according to system policy." It changes the authority of the command, not the correctness of your idea. If your path is wrong, sudo can make a wrong destructive command more powerful.

Follow with me once

Create a harmless script in your sandbox:

```
printf '#!/usr/bin/env bash\nnecho safe-run\n' > hello.sh
ls -l hello.sh
```

Try:

```
./hello.sh
```

If the file lacks execute permission, Bash may report permission denied. Inspect first, then add only the permission you need:

```
chmod u+x hello.sh
ls -l hello.sh
./hello.sh
```

u+x means "add execute permission for the owner." It is much more informative than giving everyone every permission.

Compare:

```
bash hello.sh
./hello.sh
```

The first asks Bash to read the file as a script; the second asks the operating system to execute the file directly, so the execute bit and shebang matter.

For package management, understand the lifecycle before typing commands:

```
sudo apt update
apt list --upgradable 2>/dev/null | head
```

apt update refreshes package metadata; it does not mean "upgrade everything." Only install packages the current course step actually requires. Record what you installed and why.

A permission-denied recovery ladder

1. Read the exact error and path.
2. pwd and ls -l the target.
3. Ask whether you own the file and whether the operation genuinely needs elevated privileges.
4. Change the smallest relevant permission or path.
5. Use sudo only when the system-level operation legitimately requires it.

Never solve a course-project permission problem by recursively changing permissions on /, /home, or a broad Windows mount.

Try it yourself

Create DE06-04-first-attempt/run_check.sh without execute permission, capture the failed direct-execution output, inspect its mode, then add only owner execute and rerun. Write three sentences explaining why chmod 777 would be an unjustified response.

Check your work

You can read the basic rwx groups, distinguish running a script through Bash from executing it, and explain why elevated permission is a risk boundary.

Changed retry

Create a second script and make it executable with a symbolic mode change. Then remove only owner execute, prove direct execution fails, and restore it. Do not use numeric 777 at any point.

Evidence to keep

Diagnose a permission-denied script, add only owner execute, and explain why sudo/777 are not default fixes. Preserve the first attempt, observed output/error, one independent validation, and your own-words teach-back. If review is used, keep the changed retry separately.

DE06-05

Git state: init, status, add, commit, diff and log

Watch first

[1. freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\]](#)

01:04-05:41 - bounded mental-model visual only. Git command practice is taught by the learner book/lab.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Source: freeCodeCamp - Git & GitHub Crash Course for Beginners [2026] URL:

<https://www.youtube.com/watch?v=mAFoROnOfHs> Use:

01:04-05:41 - bounded mental-model visual onl Git command practice is taught by learner book/lab.

Why this matters

Git is often taught as a spell: add, commit, push. That creates confusion as soon as git status shows three kinds of state. Git becomes much easier when you understand the three local areas first: working tree, staging area/index, and committed history.

Picture it in your head

Imagine editing a report at your desk. The working tree is what is currently on the desk. The staging area is the exact set of changes you have selected for the next photograph. A commit is the photograph plus metadata and a parent link. GitHub is not part of this three-area local model; a remote comes later.

git add does not "upload." git commit does not "send to GitHub." git status is your dashboard between each step.

Follow with me once

Create a learner-owned repository:

```
mkdir -p ~/de-study/M06/git-state-lab
cd ~/de-study/M06/git-state-lab
git init
git config user.name 'Your Name'
git config user.email 'you@example.com'
printf '# Pipeline Notes\n' > README.md
git status
```

README is untracked. Stage it deliberately:

```
git add README.md
git status
git diff --staged
```

Now the next commit snapshot contains README. Commit:

```
git commit -m 'Add pipeline notes README'
git log --oneline --decorate -5
```

Modify the file:

```
printf 'Input: orders.csv\n' >> README.md
git diff
```

git diff shows working-tree changes that are not staged. Stage them, then change the file again:

```
git add README.md
printf 'Output: summary.json\n' >> README.md

git status
git diff
git diff --staged
```

This is a crucial moment: the staged snapshot and the current working file are now different. If you commit now, only the staged version enters that commit. This is why "I ran git add earlier" does not mean all later edits are staged.

Commit quality

A commit should represent one understandable change. "stuff", "final", and "update2" hide intent. A useful message here could be Document input contract. A large data-engineering project becomes debuggable when history explains decisions.

Try it yourself

Build a new repository with README.md and pipeline.py. Make one commit for the README and another for a small Python placeholder. Before each commit save: git status, git diff, and git diff --staged. Explain in your own words which content will enter each commit and which will not.

Check your work

Given any git status, you can point to working-tree, staged and committed state. You can prove a commit exists in git log and explain why a commit is not the same thing as a GitHub upload.

Changed retry

In a fresh repository, stage a file, edit it again after staging, and predict what the commit will contain. Commit, then prove the unstaged second edit remains in the working tree.

Evidence to keep

Create a repo and prove working tree -> staging -> commit state with status/diff/staged-diff/log evidence. Preserve the first attempt, observed output/error, one independent validation, and your own-words teach-back. If review is used, keep the changed retry separately.

DE06-06

Branches, merges and real conflict resolution

Watch first

[1. freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\]](#)

40:58-52:13: branches, checkout, merge and merge-conflict resolution.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Source: freeCodeCamp - Git & GitHub Crash Course for Beginners [2026] URL:

<https://www.youtube.com/watch?v=mAFoROnOfHs> Use: 40:58-52:13: branches, checkout, merge and merge-conflict resolution.

Why this matters

Branches let you develop a change without pretending the main line is already correct. Merge conflicts are not Git "breaking." A conflict means Git cannot safely decide how to combine competing edits. Your job is to understand the intended final state, not merely remove the scary markers.

Picture it in your head

A branch name is a movable label pointing at a commit. When you commit on feature/quality, that label moves forward. main stays where it was until you merge or otherwise move it. A merge combines histories. If both sides changed the same nearby lines differently, Git pauses and asks you to resolve the content.

Follow with me once

Use a disposable lab repository. Start with:

```
stage=raw
```

Commit it on main. Create and switch to a branch:

```
git switch -c feature/quality
```

Change the same line to:

```
stage=quality
```

Commit. Switch back to main and change it instead to:

```
stage=validated
```

Commit. Now:

```
git merge feature/quality
```

Git should stop with a conflict. Inspect:

```
git status
cat pipeline.txt
```

You will see conflict markers similar to <<<<<<, =====, and >>>>>>. These markers are evidence of two competing versions. Do not choose a side just because it is labelled HEAD. Decide the business-intended final state, for example:

```
stage=validated_quality
```

Remove the markers by editing the file to that deliberate state, then:

```
git add pipeline.txt

git status
git commit -m 'Resolve pipeline stage conflict'
```

The important evidence is not "merge succeeded." It is: what did each side mean, what final content did you choose, and why?

Fast-forward vs merge commit

If main has not moved since your branch split, Git may fast-forward the label instead of creating a merge commit. That is normal. Do not force a conflict merely to get a "more advanced" history. In this lesson we deliberately create one conflict only so you can learn the recovery process safely.

Try it yourself

Create a fresh repo with a small config.txt. Make a feature branch and main branch alter the same setting differently. Save pre-merge log, conflict output, conflicted file, your reasoning note, resolved file, and post-merge log.

Check your work

You can explain branch as a pointer, not a full copy; show which branch you are on; distinguish fast-forward from conflict; and resolve conflict content based on intended behavior.

Changed retry

Create a conflict in a README run command. One branch says python app.py; the other says python3 app.py --input data.csv. Resolve it to the command that matches the actual project interface, then prove the command works. The answer must come from the project contract, not from choosing "ours" or "theirs" mechanically.

Evidence to keep

Create and resolve a real same-line conflict based on intended final behavior; preserve before/after evidence. Preserve the first attempt, observed output/error, one independent validation, and your own-words teach-back. If review is used, keep the changed retry separately.

DE06-07

GitHub remotes: push, fetch, pull and tracking

Watch first

1. freeCodeCamp - Git & GitHub Crash Course for Beginners [2026]

55:50-1:00:24 - bounded push/fetch/pull visual. Local-vs-remote reasoning is taught by the learner book/lab.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Source: freeCodeCamp - Git & GitHub Crash Course for Beginners [2026] URL:

<https://www.youtube.com/watch?v=mAFoROnOfHs> Use:

55:50-1:00:24 - bounded push/fetch/pull visual. Local-vs-remote reasoning is taught by learner book/lab.

Why this matters

A remote repository introduces a second copy of project history. Many beginners treat push, fetch and pull as synonyms for "sync." They are not. You need to know which direction data moves and whether your current branch is changed automatically.

Picture it in your head

Your local repository has local branches such as main. A remote named origin is just a configured address. Git also keeps remote-tracking references such as origin/main, which record the remote state you last learned about.

- push: send local commits to a remote branch (subject to permissions/rules).
- fetch: download remote commits/references; it does not merge them into your current
- branch.
- pull: fetch and then integrate according to configured behavior. Because it combines steps,
- you should know what you expect before using it.

Follow with me once - no GitHub account required

The supplied practice/run_local_git_lab.sh creates a bare local repository that acts like a remote, pushes to it, and clean-clones from it. This proves Git remote mechanics without internet credentials. Run it from a Linux shell:

```
bash practice/run_local_git_lab.sh ~/de-study/M06/local-remote-lab
```

When it prints LOCAL_GIT_LAB_PASS, inspect the learner clone:

```
cd ~/de-study/M06/local-remote-lab/learner
git remote -v
git branch -vv
git log --oneline --decorate --graph --all -10
```

The remote is local, but the concepts are real: origin points elsewhere, your branch can track a remote branch, and a clean clone reconstructs committed/tracked state.

Follow with me once - GitHub mode

When you use your own GitHub repository, create an empty repository or follow the exact GitHub UI state you actually see. Add it as a remote only after verifying the URL:

```
git remote add origin <your-repository-url>
git remote -v
git push -u origin main
```

Do not paste credentials into a remote URL, README or terminal transcript. Authentication methods can change; follow GitHub's current authentication UI/docs.

To understand fetch, make a remote-side change through another clone or GitHub, then:

```
git fetch origin
git status -sb
git log --oneline --decorate --all -8
```

Your local main does not silently become the fetched commit just because origin/main moved.

Try it yourself

Use either your own GitHub repo plus the local lab, or the local lab alone if credentials/network are unavailable. Produce evidence for remote -v, tracking branch, one push, one fetch where you explain what changed, and one clean clone. Label offline/local simulation honestly.

Check your work

You can explain origin, local branch, remote-tracking branch, push, fetch and pull without saying "they all sync." You never fabricate a GitHub push screenshot when you only used a local bare remote.

Changed retry

Use a second bare remote and two clones. Make a commit in clone A, push it, then fetch from clone B. Before merging/pulling in B, prove the working file has not changed while origin/main has advanced.

Evidence to keep

Prove local/remote/tracking state using a local bare remote and real GitHub only if actually available. Preserve the first attempt, observed output/error, one independent validation, and your own-words teach-back. If review is used, keep the changed retry separately.

DE06-08

Pull requests and review workflow

Watch first

[1. MIT Missing Semester - Version control \(Git\), embedded lecture](#)

Optional visual - no verified bounded range. Focus on repositories, commits, branches and remotes. Pause to inspect your own Git history; practise destructive-looking recovery examples only in the disposable course repository.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Source: GitHub Docs - Pull request quickstart URL:

<https://docs.github.com/en/pull-requests/get-started/pull-request-quickstart> Use:

Current GitHub PR workflow reference; learner book/lab own review evidence. Offline fallback is explicitly allowed.

Why this matters

A pull request is where version control becomes a team decision. It is not "another Git command." It is a review object hosted by platforms such as GitHub: proposed branch changes, their diff, discussion, checks and merge decision.

Picture it in your head

Imagine main as a protected published route and your feature branch as a proposed detour. A pull request says: "Here is the exact diff I want to merge, here is why, please review it." Reviewers can comment, approve or request changes. Further commits to the branch update the PR.

GitHub's current quickstart follows the same chain you should learn: branch/fork -> edit -> commit -> push -> open PR -> request review -> address feedback -> merge or close.

Follow with me once

On a small repository, create a branch with one focused change:

```
git switch -c feature/add-validation-note
# edit README.md
git add README.md
git commit -m 'Document validation control'
git push -u origin feature/add-validation-note
```

Before opening the PR, inspect locally:

```
git diff main...HEAD
```

Your PR description should answer:

1. What changed?
2. Why is it needed?

3. How did you validate it?
4. What risk or limitation remains?

Do not write "please merge" with no evidence.

In GitHub, inspect the Files changed tab. A PR is partly a human-readable review of that diff. If review finds a problem, edit locally, commit a repair, and push the same branch; GitHub updates the PR. You do not need a new PR for each repair to that proposal.

Review decisions

Current GitHub reviews distinguish general comment, approve, and request changes. A self-review is not equivalent to an independent teammate review, but for a solo portfolio it is still useful to write a deliberate review note that identifies one risk and verifies the diff. Never claim "peer reviewed" unless another person actually reviewed it.

Try it yourself

If GitHub is available, open a PR for a small M06 repository change. Save the PR URL, description, diff summary, one review/comment event, any repair commit, and final merge/close decision. If GitHub is unavailable, build a local "PR simulation" note from git diff main...feature and clearly label it OFFLINE REVIEW PRACTICE - NOT A GITHUB PR.

Check your work

You can distinguish branch, commit, push and PR; explain why a PR is useful even when Git can merge locally; and prove that review feedback maps to a specific diff/commit.

Changed retry

Review a changed branch where .env was accidentally added. Do not merge. Explain the required response: remove/stop tracking the file, protect future tracking, and rotate any real credential if it was exposed. Then verify the proposed diff no longer contains the secret file.

Evidence to keep

Open/review a real PR when available or honestly labelled offline review practice; connect feedback to specific diff. Preserve the first attempt, observed output/error, one independent validation, and your own-words teach-back. If review is used, keep the changed retry separately.

DE06-09

Python virtual environments inside a repository

Watch first

[1. Corey Schafer - Virtual environments for Mac and Linux](#)

Optional visual - no verified bounded range. Use the Linux commands inside WSL. Activate the repository environment before installing or running Python dependencies.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Source: [Python 3.14 docs - venv URL: https://docs.python.org/3/library/venv.html](https://docs.python.org/3/library/venv.html) Use: Current virtual-environment behavior: isolated packages, disposable/recreatable environments, create with `python -m venv`.

Why this matters

A repository should describe a project, not depend on whichever Python packages happen to be installed globally on your laptop. A virtual environment gives the project an isolated package environment that can be deleted and recreated.

Picture it in your head

Your source code belongs in the repository. `.venv` is a local tool-room built for that project. It may contain many generated files and machine-specific paths; it is not the project itself. Python's current documentation explicitly treats virtual environments as disposable/recreatable and not something to check into source control.

Follow with me once

Inside a project repository:

```
python3 --version
python3 -m venv .venv
. .venv/bin/activate
python --version
python -m pip --version
```

The shell prompt often changes, but do not use prompt decoration as your only proof. Inspect:

```
python -c 'import sys; print(sys.executable)'
```

It should point inside `.venv`.

Add `.venv/` to `.gitignore` and verify:

```
printf '.venv/\n' >> .gitignore
git status --short
git check-ignore -v .venv/pyvenv.cfg
```

The environment can be rebuilt; your repository should record required dependencies/configuration through appropriate project files, not ship your personal environment directory.

For this module's standard-library sample project, requirements.txt deliberately contains no third-party dependency. That is valid: do not install random packages merely to make the file non-empty.

Activation is convenience, not magic

Activation modifies shell environment so python/pip resolve to the virtual environment. You can also call its interpreter explicitly. The important question is which interpreter you are using, not whether the prompt shows parentheses.

Try it yourself

In a fresh repository, create .venv, prove its interpreter path, add the ignore rule, and demonstrate that .venv is not staged by git add .. Deactivate, delete .venv, recreate it, and prove the project still runs using only documented steps.

Check your work

You can explain why .venv should not be committed, prove which interpreter is active, and recreate the environment from repository instructions.

Changed retry

Create a second project environment named .venv, install no third-party package, then show that deleting the environment does not delete your source. Recreate it and run the same script from a fresh shell.

Evidence to keep

Create, ignore, delete and recreate .venv; prove project source remains reproducible. Preserve the first attempt, observed output/error, one independent validation, and your own-words teach-back. If review is used, keep the changed retry separately.

DE06-10

.gitignore, secrets and repository hygiene

Watch first

[1. freeCodeCamp - Git & GitHub Crash Course for Beginners \[2026\]](#)

OPTIONAL visual - no verified bounded range. Use only for Git status/tracking/removal context; learner book owns .gitignore and secret-response policy.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Source: freeCodeCamp - Git & GitHub Crash Course for Beginners [2026] URL:
<https://www.youtube.com/watch?v=mAFoROnOfHs> Use:

OPTIONAL visual - no verified bounded range; Git hygiene context only.

Why this matters

Repositories are easy to copy, fork and publish. A secret committed once can escape far beyond your laptop. .gitignore helps prevent unwanted untracked files from entering future commits, but it is not a time machine and not a credential-revocation system.

Picture it in your head

Think of .gitignore as a doorman checking untracked files before Git starts tracking them. If a file is already tracked, adding its name to .gitignore does not make Git forget its history. If a real password/token has been exposed, removing the file from the next commit does not make that credential safe again; rotate/revoke it.

Follow with me once

A basic project ignore file might contain:

```
.env/  
__pycache__/  
.env  
output/  
*.log
```

Create a fake, clearly non-secret .env only for the lab:

```
printf 'DEMO_TOKEN=not-a-real-secret\n' > .env  
git status --short  
git check-ignore -v .env
```

It should be ignored. Create .env.example with variable names/placeholders only and intentionally track that:

```
printf 'DEMO_TOKEN=replace_me_locally\n' > .env.example  
git add .gitignore .env.example  
git diff --staged
```

Before every push, `git status` and `git diff --staged` are cheap secret-hygiene controls.

The already-tracked trap

If a harmless demo file was already committed, adding it to `.gitignore` will not stop Git from tracking it. `git rm --cached <file>` can remove it from the current tracked set while keeping a local copy, but historical commits still exist. For a real secret, your first operational response is credential rotation/revocation. History cleaning is a separate incident-handling task and may require coordination.

Never paste real tokens into screenshots, terminal transcripts, course answers or AI prompts for "debugging."

Try it yourself

Create a repository with `.venv/`, `output/`, `.env`, `.env.example`, and one log file. Write ignore rules so only `.env.example` is eligible to commit. Use `git status --ignored --short` and `git check-ignore -v` to prove why each ignored file is ignored. Save staged diff evidence before commit.

Check your work

You can explain what `.gitignore` does and does not do, prove ignored state, and state the correct response to an exposed real credential: rotate/revoke first, then repair repository/history as required.

Changed retry

Start with a harmless `demo-secret.txt` intentionally tracked in one disposable commit. Add it to `.gitignore` and observe that it remains tracked. Stop tracking it from the current tree, commit the repair, then use `git log --all -- demo-secret.txt` to prove why "it disappeared from the latest files" is not the same as "it never existed in history."

Evidence to keep

Prove ignored-vs-tracked behavior and safe secret response using only fake lab values. Preserve the first attempt, observed output/error, one independent validation, and your own-words teach-back. If review is used, keep the changed retry separately.

DE06-11

restore, revert, reflog and safe history repair

Watch first

1. MIT Missing Semester - Version control (Git), embedded lecture

Optional visual - no verified bounded range. Focus on repositories, commits, branches and remotes. Pause to inspect your own Git history; practise destructive-looking recovery examples only in the disposable course repository.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Source: Official Git docs - reset, restore, revert and reflog URL: <https://git-scm.com/docs/git> Use: Use the official distinction between restore/revert/reset; pair with git-reflog docs. Course lab focuses safe recovery rather than history tricks.

Why this matters

Undo commands are dangerous when chosen by memory instead of state. A professional asks two questions first: Is the bad change committed? and Has the commit been shared/published? The safest tool depends on that answer.

Picture it in your head

- git restore <file>: repair working-tree content from a selected source; it does not create a new history commit by itself.
- git revert <commit>: create a new commit that reverses an earlier commit. This is often appropriate for published/shared history because it preserves the record.
- git reflog: your local record of where references/HEAD recently pointed; useful when you think a commit was "lost." Reflogs are local, not a remote backup.
- git reset can move branch history and may discard state depending on mode. It is not our default beginner repair tool.

Follow with me once - restore

Modify a tracked file incorrectly but do not commit it:

```
git status
git diff -- README.md
git restore README.md
git status
```

You intentionally discard that uncommitted modification. This is why you inspect the diff first: restore can remove work you wanted.

Follow with me once - revert

Make a disposable bad commit, for example changing a threshold from 100 to 999. Save its hash:

```
git log --oneline -3
```

Then:

```
git revert <bad-commit-hash>
```

Inspect the new log. The bad commit still exists, followed by a new commit that reverses its patch. History tells the truth about what happened.

Follow with me once - reflog

Switch branches and make several commits, then inspect:

```
git reflog --oneline -10
```

Reflog entries show recent HEAD/reference movement. If you accidentally moved a branch and can no longer see a commit in ordinary git log, reflog can help identify the old commit ID so you can create a recovery branch/tag before doing anything else.

Recovery decision before command

1. Save current git status, diff and log/reflog evidence.
2. Decide whether state is uncommitted, locally committed, or shared.
3. Choose the narrowest repair.
4. Re-run the project or validation after repair.
5. Explain what history changed.

Try it yourself

In a disposable repository, prove all three: restore an uncommitted change after inspecting it; create/revert a bad commit; and use reflog to locate a prior HEAD position. Save before/after logs. Do not use reset --hard in the exercise.

Check your work

You can choose restore vs revert based on state and explain reflog as a local recovery record, not "Git backup in the cloud."

Changed retry

Create a new bad commit on a feature branch, pretend it was already shared, and use revert rather than rewriting it away. Then deliberately detach/switch around and use reflog to identify an earlier commit, creating recovery-check branch at that commit without deleting current work.

Evidence to keep

Demonstrate restore, revert and reflog recovery in a disposable repository without reset --hard. Preserve the first attempt, observed output/error, one independent validation, and your own-words teach-back. If review is used, keep the changed retry separately.

DE06-12

Shell scripts, exit codes and environment-driven commands

Watch first

[1. freeCodeCamp - Bash Scripting Tutorial for Beginners](#)

OPTIONAL visual - no verified bounded range. Use only for Bash scripting context; learner book + shell lab are required.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Source: freeCodeCamp - Bash Scripting Tutorial for Beginners URL:

<https://www.youtube.com/watch?v=tK9Oc6AEnR4> Use:

OPTIONAL visual - no verified bounded range. Use only for Bash context; book + shell lab required.

Why this matters

A shell script turns a remembered terminal sequence into an executable contract. Data pipelines and deployment jobs use exit codes to decide whether a step succeeded. A script that prints "error" but exits zero can fool an orchestrator into treating failure as success.

Picture it in your head

A script is a small program run by a shell. The shebang identifies the intended interpreter when the script is executed directly. Environment variables carry external configuration. Positional arguments provide invocation-specific input. Exit code 0 conventionally signals success; non-zero signals failure.

Follow with me once

Inspect the supplied practice/run_exit_code_lab.sh:

```
bash practice/run_exit_code_lab.sh
printf 'exit=%s\n' "$?"
```

With no argument it should fail non-zero. Now:

```
bash practice/run_exit_code_lab.sh data/linux_orders.csv
printf 'exit=%s\n' "$?"
```

It should print a line count and exit 0.

A robust small wrapper can start with:

```
#!/usr/bin/env bash
set -euo pipefail
```

But do not copy set -euo pipefail as superstition. Roughly: -e stops on many unhandled command failures, -u treats unset variables as errors, and pipefail lets a pipeline report failure if an earlier

stage failed. Shell edge cases exist, so you still need explicit validation around important boundaries.

Use variables safely:

```
input="${1:-}"
if [ -z "$input" ]; then
    echo 'usage: run.sh INPUT_FILE' >&2
    exit 2
fi
```

Quote variable expansions such as "\$input" so spaces/globbering do not silently turn one path into several arguments.

Environment-driven configuration example:

```
: "${APP_ENV:=development}"
echo "environment=$APP_ENV"
```

Do not print secret variables for debugging.

Try it yourself

Complete practice/M06_Bash_Wrapper_TODO.sh in your own copy. It must detect missing configuration/input, invoke the sample Python project, return non-zero on failure, and print a short safe success summary. Test success, missing variable, and missing-file paths. Record each exit code.

Check your work

A calling script can tell success from failure using the exit status; your wrapper quotes paths; no secret is printed; and a missing required input fails before downstream work begins.

Changed retry

Write a second wrapper that accepts shipments.csv as a positional argument and an optional output directory through OUTPUT_DIR. Test a filename containing a space. Show one failing and one successful exit code.

Evidence to keep

Complete a Bash wrapper with arguments/environment, quoted paths and meaningful exit codes. Preserve the first attempt, observed output/error, one independent validation, and your own-words teach-back. If review is used, keep the changed retry separately.

DE06-13

Repository handoff and clean-clone reproducibility

Watch first

[1. MIT Missing Semester - Version control \(Git\), embedded lecture](#)

Optional visual - no verified bounded range. Focus on repositories, commits, branches and remotes. Pause to inspect your own Git history; practise destructive-looking recovery examples only in the disposable course repository.

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Source: GitHub Docs + freeCodeCamp Git course URL:

<https://docs.github.com/en/repositories/creating-and-managing-repositories/cloning-a-repository>

Use: Clone into a genuinely fresh folder. Course defines the reproducibility/handoff proof and forbids relying on undeclared local files.

Why this matters

A repository is not truly reproducible because it runs on the laptop where it was created. The strongest beginner test is intentionally inconvenient: clone it into a fresh directory and follow only the documented instructions. Hidden local files, global packages, undeclared environment variables and path assumptions become visible immediately.

Picture it in your head

Imagine handing the repository to an interviewer who has never seen your computer. They should not need your Downloads folder, your global Python site-packages, a secret hidden in shell history or an instruction you forgot to write down. The repository should state the start point, prerequisites, setup, run command, inputs, outputs and validation.

Follow with me once

Take the supplied practice/sample_project as a model only for handoff mechanics, not as your final evidence. Put your learner-owned project in a Git repository and push to a remote (GitHub when available, local bare remote is acceptable for mechanics). Then leave the original folder.

Clone to a genuinely different path:

```
cd ~/de-study/M06
rm -rf clean-check # ONLY if this exact disposable folder is yours and inspected
git clone <remote-url-or-local-bare-path> clean-check
cd clean-check
```

Do not copy your old .venv. Follow README:

```
python3 -m venv .venv
. .venv/bin/activate
python app.py
```

For the supplied sample project the independent control is:

```
source_rows = 6
completed_rows = 4
completed_amount = 3900
```

Also verify ignored/generated state:

```
git status --short
```

A clean-clone run that requires untracked source data from your old machine is a failed handoff even if the original project still runs.

The handoff defense

You should be able to answer:

- What is the entry point?
- Which files are source vs generated?
- What environment/config must be supplied?
- What should never be committed?
- What exact output/control proves the run worked?
- What happens on invalid/missing input?

You do it alone - M06 module practical

Complete practice/M06_Checkpoint_TODO.md. Use a learner-owned repository. Preserve first failed attempts. If GitHub is available, include real remote/PR evidence; if not, label local remote mechanics honestly and leave GitHub runtime evidence pending rather than inventing it.

Check your work

A clean clone can be set up and run from README only, the expected controls reconcile, ignored/generated files remain untracked, and you can explain the complete Git/Linux/reproducibility path in your own words.

Changed retry

After review, complete practice/M06_Fresh_Checkpoint_TODO.md in a new repository with changed file names, shell interface, branch name, conflict content and revert scenario. Do not reuse your first repo by merely renaming it.

Evidence to keep

Perform clean-clone handoff from README only and defend the repository; then do the changed checkpoint. Preserve the first attempt, observed output/error, one independent validation, and your own-words teach-back. If review is used, keep the changed retry separately.